Métriques de Précision et d'Erreur du Modèle YOLOv8

Détection et Reconnaissance d'Objets

Référence exhaustive — Définitions, formules, calculs détaillés et exemples numériques sur des données réelles

April 25, 2026

Contents

1 Introduction **3**

1.1 Contexte et objectifs 3

1.2 Notions fondamentales : TP, FP, FN, TN 3

2 Métriques de Précision et de Rappel **5**

2.1 Precision (P) 5

2.2 Recall (R) — Rappel 6

2.3 F1-Score 7

2.4 AP — Average Precision 7

2.4.1 Construction de la courbe PR 8

2.5 mAP50 — mean Average Precision @ IoU=0.50 9

2.6 mAP50-95 — mean Average Precision @ IoU=0.50:0.95 9

3 Métriques de Localisation — Bounding Box **11**

3.1 IoU — Intersection over Union 11

3.2 GIoU — Generalized IoU 12

3.3 DIoU — Distance IoU 13

3.4 CIoU — Complete IoU (Loss par défaut de YOLOv8) 14

4 Fonctions de Perte (Loss Functions) **16**

4.1 Box Loss 16

4.2 Classification Loss (Cls Loss) 17

4.3 DFL Loss — Distribution Focal Loss 18

5 Métriques Opérationnelles **19**

5.1 FPS — Frames Per Second 19

5.2 GFLOPs — Giga Floating Point Operations 20

5.3 Paramètres du modèle (Params) 20

6 Métriques de Diagnostic **21**

6.1 Confidence Score 21

6.2 Courbe PR (Precision-Recall) 22

6.3 Courbe F1-Confidence 22

6.4 Matrice de Confusion 23

7 Tableau Récapitulatif Exhaustif **24**

8 Implémentation Python de Référence **26**

9 Conclusion et Guide Pratique **29**

9.1 Quelle métrique utiliser selon l'objectif ? 29

9.2 Seuils de référence YOLOv8 (COCO) 29

9.3 Checklist d'évaluation YOLOv8 30

Chapter 1

Introduction

1.1 Contexte et objectifs

YOLOv8 (*You Only Look Once version 8*), développé par Ultralytics, est l'une des architectures les plus performantes pour la **détection et la reconnaissance d'objets** en temps réel. Évaluer correctement un tel modèle requiert de maîtriser un ensemble de métriques couvrant quatre dimensions complémentaires :

- **Précision de la classification** : le modèle attribue-t-il la bonne étiquette ?
- **Qualité de la localisation** : la boîte englobante est-elle bien positionnée ?
- **Erreurs d'entraînement** : les fonctions de perte convergent-elles correctement ?
- **Performance opérationnelle** : le modèle peut-il tourner en temps réel ?

1.2 Notions fondamentales : TP, FP, FN, TN

Définitions de base

Soit une image contenant des objets réels (Ground Truth, GT). Le modèle produit des **prédictions**. On compare chaque prédiction au GT via un seuil d'IoU (typiquement 0,50) :

- **TP (True Positive)** : la prédiction correspond à un objet réel et la classe est correcte, *et* IoU $\geq$ seuil.
- **FP (False Positive)** : la prédiction ne correspond à aucun objet réel, ou la classe est incorrecte.

- **FN (False Negative)** : un objet réel n'a pas été détecté.
- **TN (True Negative)** : zone sans objet, non détectée (rarement utilisé en détection).

Exemple illustratif — 5 images, classe "voiture"

Image	GT réels	Détections	TP	FP	FN
Img-1	2	2	2	0	0
Img-2	3	4	3	1	0
Img-3	1	0	0	0	1
Img-4	2	2	1	1	1
Img-5	0	1	0	1	0
Total	**8**	**9**	**6**	**3**	**2**

Les données de cet exemple (TP = 6, FP = 3, FN = 2) seront réutilisées dans tout le document.

Chapter 2

Métriques de Précision et de Rappel

2.1 Precision (P)

Définition

La **Precision** mesure la proportion de détections correctes parmi *toutes* les détections émises par le modèle. Elle répond à la question : Quand le modèle dit qu'il a trouvé un objet, a-t-il raison ?

Formule

$$P = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

- TP : nombre de vraies détections correctes
- FP : nombre de fausses alarmes (détections erronées)

Interprétation :

- $P = 1{,}0$: toutes les détections sont correctes, aucune fausse alarme.
- $P = 0{,}5$: une détection sur deux est fausse.
- Une précision faible est dangereuse dans des systèmes de surveillance (trop d'alertes inutiles).

Calcul numérique

En reprenant les données du tableau (TP = 6, FP = 3) :

$$P = \frac{6}{6+3} = \frac{6}{9} \approx \mathbf{0{,}667}$$

Interprétation : 66,7 % des détections émises par le modèle correspondent effectivement à des voitures réelles.

2.2 Recall (R) — Rappel

Définition

Le **Recall** mesure la proportion d'objets réels effectivement détectés parmi tous ceux présents dans les images. Il répond à la question : Le modèle manque-t-il des objets ?

Formule

$$R = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

- FN : objets réels non détectés

Interprétation :

- $R = 1{,}0$: aucun objet n'a été manqué.
- $R = 0{,}5$: la moitié des objets n'est pas détectée.
- Un rappel faible est critique dans des applications médicales ou de sécurité (un objet manqué peut avoir de graves conséquences).

Calcul numérique

Avec TP = 6 et FN = 2 :

$$R = \frac{6}{6+2} = \frac{6}{8} = \mathbf{0{,}75}$$

Interprétation : le modèle détecte 75 % des voitures réellement présentes dans les images.

Compromis Precision / Recall

Il existe un **trade-off fondamental** : augmenter le seuil de confiance améliore la Precision (moins de fausses alarmes) mais réduit le Recall (plus d'objets manqués), et vice-versa. Le F1-Score et la courbe PR permettent d'analyser ce compromis.

2.3 F1-Score

Définition

Le **F1-Score** est la moyenne harmonique de la Precision et du Recall. Il fournit un indicateur unique équilibrant les deux métriques précédentes.

Formule

$$F1 = 2 \times \frac{P \times R}{P + R}$$

Équivalent développé :

$$F1 = \frac{2\,\mathrm{TP}}{2\,\mathrm{TP} + \mathrm{FP} + \mathrm{FN}}$$

Pourquoi la moyenne harmonique ?

La moyenne arithmétique $(P+R)/2$ peut donner un score élevé même si l'une des deux métriques est très faible. Par exemple, si $P = 1{,}0$ et $R = 0{,}1$:

$$\text{Moy. arith.} = \frac{1{,}0 + 0{,}1}{2} = 0{,}55 \quad \text{vs} \quad F1 = \frac{2 \times 1{,}0 \times 0{,}1}{1{,}0 + 0{,}1} \approx 0{,}18$$

La moyenne harmonique pénalise fortement les déséquilibres.

Calcul numérique

Avec $P = 0{,}667$ et $R = 0{,}75$:

$$F1 = 2 \times \frac{0{,}667 \times 0{,}75}{0{,}667 + 0{,}75} = 2 \times \frac{0{,}500}{1{,}417} \approx \mathbf{0{,}706}$$

Vérification directe : $F1 = \frac{2\times 6}{2\times 6+3+2} = \frac{12}{17} \approx 0{,}706$ ✓

Utilisation en YOLOv8 : la courbe F1-Confidence permet de sélectionner le seuil de confiance optimal (celui qui maximise le F1-Score sur les données de validation).

2.4 AP — Average Precision

Définition

L'**Average Precision** est l'aire sous la courbe Precision-Recall (PR). Elle synthétise la performance du modèle sur une *classe donnée* sur *tous les seuils de confiance possibles.*

Formule (interpolation 11 points)

$$AP = \frac{1}{11} \sum_{r \in \{0, 0.1, \ldots, 1.0\}} P_{\text{interp}}(r)$$

où $P_{\text{interp}}(r) = \max_{\tilde{r} \geq r} P(\tilde{r})$

Ou, en intégrale continue (COCO) :

$$AP = \int_0^1 P(R)\, dR$$

2.4.1 Construction de la courbe PR

1. Trier toutes les détections par score de confiance décroissant.
2. Pour chaque seuil de confiance c_i, calculer $P(c_i)$ et $R(c_i)$.
3. Tracer la courbe $P = f(R)$.
4. Calculer l'aire sous cette courbe.

Calcul détaillé de l'AP

Supposons 10 détections triées par confiance décroissante pour la classe "voiture" (GT total = 5 objets). "1" = TP, "0" = FP :

Rang	Conf.	TP/FP	Precision	Recall
1	0.95	1	$1/1 = 1.000$	$1/5 = 0.200$
2	0.90	1	$2/2 = 1.000$	$2/5 = 0.400$
3	0.85	0	$2/3 = 0.667$	$2/5 = 0.400$
4	0.80	1	$3/4 = 0.750$	$3/5 = 0.600$
5	0.75	0	$3/5 = 0.600$	$3/5 = 0.600$
6	0.70	1	$4/6 = 0.667$	$4/5 = 0.800$
7	0.65	0	$4/7 = 0.571$	$4/5 = 0.800$
8	0.60	1	$5/8 = 0.625$	$5/5 = 1.000$
9	0.50	0	$5/9 = 0.556$	$5/5 = 1.000$
10	0.40	0	$5/10 = 0.500$	$5/5 = 1.000$

Après interpolation (valeur max de P pour $R \geq r$) aux 11 points :

$$AP = \frac{1}{11}(1{,}0 + 1{,}0 + 1{,}0 + 0{,}75 + 0{,}75 + 0{,}667 + 0{,}625 + 0{,}625 + 0 + 0 + 0) \approx \mathbf{0{,}584}$$

2.5 mAP50 — mean Average Precision @ IoU=0.50

Définition

Le **mAP50** est la moyenne des AP de toutes les classes, calculée avec un seuil IoU fixé à 0,50. C'est la **métrique principale** affichée par YOLOv8 lors de l'évaluation.

Formule

$$\text{mAP}_{50} = \frac{1}{N} \sum_{i=1}^{N} AP_i^{(\text{IoU}=0.50)}$$

où N est le nombre de classes.

Calcul numérique — 4 classes

Classe	AP_{50}	Observations
Voiture	0.892	Classe bien représentée
Piéton	0.741	Petits objets, plus difficile
Vélo	0.654	Souvent occulté
Camion	0.813	Grands objets, bien détectés
mAP50	**0.775**	$(0.892 + 0.741 + 0.654 + 0.813)/4$

Interprétation : le modèle obtient en moyenne 77,5 % de précision sur les 4 classes avec un seuil de recoupement de 50 %.

2.6 mAP50-95 — mean Average Precision @ IoU=0.50:0.95

Définition

Le **mAP50-95** est la moyenne des mAP calculés sur 10 seuils IoU distincts : $\{0.50, 0.55, 0.60, \ldots, 0.95\}$. C'est la métrique standard du challenge **COCO**, la plus exigeante.

Formule

$$\text{mAP}_{50:95} = \frac{1}{10} \sum_{t \in \{0.50,\, 0.55,\, \ldots,\, 0.95\}} \text{mAP}_t$$

Calcul numérique

Seuil IoU	**mAP$_t$**
0.50	0.775
0.55	0.742
0.60	0.708
0.65	0.671
0.70	0.623
0.75	0.561
0.80	0.489
0.85	0.401
0.90	0.283
0.95	0.112
mAP50-95	**0.537**

La chute des valeurs montre que le modèle se dégrade significativement quand on exige une localisation plus précise (IoU $\geq$ 0,75).

mAP50 vs mAP50-95

Un modèle peut afficher un excellent mAP50 ($\sim$ 0,90) mais un mAP50-95 médiocre ($\sim$ 0,45), indiquant que les boîtes englobantes sont imprécises. Pour des applications critiques (robotique, véhicules autonomes), le mAP50-95 est la métrique pertinente.

Chapter 3

Métriques de Localisation — Bounding Box

3.1 IoU — Intersection over Union

Définition

L'**IoU** mesure le taux de recoupement entre la boîte prédite B_{pred} et la boîte réelle B_{gt}. C'est la base de toutes les métriques de localisation.

Formule

$$\text{IoU} = \frac{|B_{\text{pred}} \cap B_{\text{gt}}|}{|B_{\text{pred}} \cup B_{\text{gt}}|}$$

Développé :

$$\text{IoU} = \frac{\text{Aire d'intersection}}{\text{Aire}(B_{\text{pred}}) + \text{Aire}(B_{\text{gt}}) - \text{Aire d'intersection}}$$

Calcul de l'intersection :

Soient $B_{\text{pred}} = (x_1^p, y_1^p, x_2^p, y_2^p)$ et $B_{\text{gt}} = (x_1^g, y_1^g, x_2^g, y_2^g)$ en coordonnées coin-supérieur-gauche / coin-inférieur-droit.

$$x_{\text{inter},1} = \max(x_1^p, x_1^g), \quad x_{\text{inter},2} = \min(x_2^p, x_2^g)$$

$$y_{\text{inter},1} = \max(y_1^p, y_1^g), \quad y_{\text{inter},2} = \min(y_2^p, y_2^g)$$

$$\text{Aire}_{\text{inter}} = \max(0,\ x_{\text{inter},2} - x_{\text{inter},1}) \times \max(0,\ y_{\text{inter},2} - y_{\text{inter},1})$$

Calcul numérique

- $B_{\text{pred}} = (100,\ 150,\ 250,\ 350)$ — Aire $= 150 \times 200 = 30\,000$
- $B_{\text{gt}} = (120,\ 160,\ 270,\ 370)$ — Aire $= 150 \times 210 = 31\,500$

Intersection :

$$x_1 = \max(100, 120) = 120,\ x_2 = \min(250, 270) = 250 \ \Rightarrow\ \Delta x = 130$$

$$y_1 = \max(150, 160) = 160,\ y_2 = \min(350, 370) = 350 \ \Rightarrow\ \Delta y = 190$$

$$\text{Aire}_{\text{inter}} = 130 \times 190 = 24\,700$$

Union :

$$\text{Aire}_{\text{union}} = 30\,000 + 31\,500 - 24\,700 = 36\,800$$

$$\text{IoU} = \frac{24\,700}{36\,800} \approx \mathbf{0{,}671}$$

La boîte prédite chevauche 67,1 % de la boîte réelle. Avec un seuil de 0,50, cette détection est comptée comme **TP**.

3.2 GIoU — Generalized IoU

Définition

Le **GIoU** étend l'IoU en pénalisant la distance entre les boîtes lorsqu'elles *ne se chevauchent pas*, résolvant le problème des gradients nuls de l'IoU standard.

Formule

$$\text{GIoU} = \text{IoU} - \frac{|C \setminus (B_{\text{pred}} \cup B_{\text{gt}})|}{|C|}$$

où C est la **plus petite boîte englobante convexe** contenant B_{pred} et B_{gt}.

Propriétés :

$$-1 \leq \text{GIoU} \leq 1$$

- $\text{GIoU} = 1$: correspondance parfaite.
- $\text{GIoU} \to -1$: boîtes très éloignées.
- $\text{GIoU} = \text{IoU}$ si les boîtes se chevauchent entièrement.

Calcul numérique — boîtes sans chevauchement

- $B_{\text{pred}} = (0,\ 0,\ 100,\ 100)$ — Aire $= 10\,000$
- $B_{\text{gt}} = (150,\ 150,\ 250,\ 250)$ — Aire $= 10\,000$
- Intersection $= 0$ (aucun chevauchement)
- IoU $= 0$
- Boîte $C = (0,\ 0,\ 250,\ 250)$ — Aire$(C) = 62\,500$
- Aire$(C)-$ Aire(union) $= 62\,500 - 20\,000 = 42\,500$

$$\text{GIoU} = 0 - \frac{42\,500}{62\,500} = \mathbf{-0{,}68}$$

Le GIoU de $-0{,}68$ indique que les boîtes sont très éloignées, ce que l'IoU $= 0$ ne pouvait pas quantifier.

3.3 DIoU — Distance IoU

Définition

Le **DIoU** ajoute une pénalité sur la **distance entre les centres** des deux boîtes, en plus de l'IoU. Il favorise une convergence plus rapide que le GIoU.

Formule

$$\text{DIoU} = \text{IoU} - \frac{\rho^2(b,\ b^{gt})}{c^2}$$

- $\rho(b, b^{gt})$: distance euclidienne entre les centres b et b^{gt} des deux boîtes.
- c : longueur de la diagonale de la plus petite boîte englobante C.

Calcul numérique

- $B_{\text{pred}} = (50,\ 50,\ 150,\ 150) \ \Rightarrow\ $ centre $b = (100, 100)$
- $B_{\text{gt}} = (80,\ 80,\ 180,\ 180) \ \Rightarrow\ $ centre $b^{gt} = (130, 130)$
- Intersection : $x_1 = 80, x_2 = 150, y_1 = 80, y_2 = 150 \ \Rightarrow\ $ Aire $= 70 \times 70 = 4\,900$
- Aire$_{\text{pred}} = 10\,000$, Aire$_{\text{gt}} = 10\,000$
- Aire$_{\text{union}} = 10\,000 + 10\,000 - 4\,900 = 15\,100$

- IoU $= 4\,900/15\,100 \approx 0{,}3245$
- $\rho^2 = (130-100)^2 + (130-100)^2 = 900 + 900 = 1\,800$
- $C = (50,\ 50,\ 180,\ 180) \ \Rightarrow\ c^2 = 130^2 + 130^2 = 33\,800$

$$\text{DIoU} = 0{,}3245 - \frac{1\,800}{33\,800} \approx 0{,}3245 - 0{,}0533 \approx \mathbf{0{,}271}$$

3.4 CIoU — Complete IoU (Loss par défaut de YOLOv8)

Définition

Le **CIoU** est la métrique de recoupement la plus complète : il intègre simultanément l'**IoU**, la **distance entre centres** et le **ratio d'aspect** (rapport largeur/hauteur). C'est la fonction de perte de localisation native de YOLOv8.

Formule

$$\text{CIoU} = \text{IoU} - \frac{\rho^2(b,\ b^{gt})}{c^2} - \alpha v$$

où :

$$v = \frac{4}{\pi^2}\left(\arctan\frac{w^{gt}}{h^{gt}} - \arctan\frac{w}{h}\right)^2$$

$$\alpha = \frac{v}{(1-\text{IoU}) + v}$$

- v : mesure la cohérence du ratio d'aspect entre prédiction et GT.
- α : coefficient de pondération adaptatif.
- w, h : largeur et hauteur de la boîte prédite.
- w^{gt}, h^{gt} : largeur et hauteur de la boîte GT.

Calcul numérique

En reprenant l'exemple précédent et ajoutant les dimensions :

- B_{pred} : $w = 100,\ h = 100$ (carré)
- B_{gt} : $w = 100,\ h = 100$ (carré)
- IoU $\approx 0{,}3245$, termes DIoU calculés ci-dessus.

$$v = \frac{4}{\pi^2}\left(\arctan\frac{100}{100} - \arctan\frac{100}{100}\right)^2 = \frac{4}{\pi^2}(0)^2 = 0$$

Les deux boîtes ayant le *même ratio d'aspect*, $v = 0$ et donc CIoU = DIoU $\approx$ **0,271**.

Cas avec ratio d'aspect différent ($B_{\text{pred}} : w = 100, h = 150$; $B_{\text{gt}} : w = 100, h = 100$) :

$$v = \frac{4}{\pi^2}\left(\arctan(1) - \arctan\tfrac{2}{3}\right)^2 = \frac{4}{\pi^2}(0{,}7854 - 0{,}5880)^2 \approx \frac{4}{9{,}87}(0{,}0390) \approx 0{,}0158$$

$$\alpha = \frac{0{,}0158}{(1 - 0{,}3245) + 0{,}0158} = \frac{0{,}0158}{0{,}6913} \approx 0{,}0229$$

$$\text{CIoU} \approx 0{,}3245 - 0{,}0533 - 0{,}0229 \times 0{,}0158 \approx \mathbf{0{,}271}$$

Chapter 4

Fonctions de Perte (Loss Functions)

Les fonctions de perte guident l'optimisation du modèle pendant l'entraînement. YOLOv8 dispose de trois pertes distinctes, affichées séparément sur les courbes d'apprentissage (train et val).

4.1 Box Loss

Définition

La **Box Loss** quantifie l'erreur de régression des coordonnées des boîtes englobantes. YOLOv8 utilise la CIoU Loss (ou DFL + CIoU selon la configuration).

Formule (CIoU Loss)

$$\mathcal{L}_{\text{box}} = 1 - \text{CIoU}(B_{\text{pred}},\, B_{\text{gt}})$$

Interprétation des courbes :

- $\mathcal{L}_{\text{box}} \to 0$: les boîtes prédites coïncident parfaitement avec les boîtes réelles.
- Si `val/box_loss` $\gg$ `train/box_loss` : surapprentissage de la localisation.

Exemple sur un batch

Soit un batch de 3 détections avec leurs CIoU :

Détection	CIoU	Box Loss	Pondération
Det-1	0.85	$1 - 0.85 = 0.15$	$\times 1.0$
Det-2	0.60	$1 - 0.60 = 0.40$	$\times 1.0$
Det-3	0.30	$1 - 0.30 = 0.70$	$\times 1.0$
Moyenne	—	**0.417**	—

4.2 Classification Loss (Cls Loss)

Définition

La **Cls Loss** (ou Classification Loss) mesure l'erreur d'attribution des classes aux objets détectés. YOLOv8 utilise la **Binary Cross-Entropy (BCE)** avec sigmoid, appliquée de façon indépendante à chaque classe (approche multi-label).

Formule

Pour chaque classe c et chaque prédiction :

$$\mathcal{L}_{\text{cls}} = -\big[y_c \log(\hat{p}_c) + (1 - y_c)\log(1 - \hat{p}_c)\big]$$

- $y_c \in \{0, 1\}$: label réel (1 si l'objet appartient à la classe c)
- $\hat{p}_c \in [0, 1]$: probabilité prédite par le modèle pour la classe c

Calcul numérique — 3 classes

Objet de classe "voiture" ($y_1 = 1$) avec probabilités prédites :

Classe	y_c	$\hat{p}_c$	$\mathcal{L}_c$
Voiture	1	0.85	$-(1 \times \log 0.85) = 0.163$
Piéton	0	0.10	$-(1 \times \log 0.90) = 0.105$
Vélo	0	0.05	$-(1 \times \log 0.95) = 0.051$
Total	—	—	**0.319**

La loss est faible car le modèle attribue correctement une haute probabilité à "voiture" et de faibles probabilités aux autres classes.

4.3 DFL Loss — Distribution Focal Loss

Définition

La **DFL Loss** est une innovation de YOLOv8 : au lieu de prédire directement les coordonnées scalaires des bords des boîtes, le modèle prédit une **distribution de probabilité** sur un ensemble discret de valeurs possibles. La DFL Loss optimise cette distribution.

Formule

$$\mathrm{DFL}(S_i,\, S_{i+1}) = -\Big[\,(y_{i+1} - y)\,\log(S_i) + (y - y_i)\,\log(S_{i+1})\,\Big]$$

- y : valeur cible du bord (continue)
- y_i, y_{i+1} : deux valeurs discrètes encadrant y (i.e. $y_i \leq y < y_{i+1}$)
- S_i, S_{i+1} : probabilités prédites pour y_i et y_{i+1} (issues du softmax)

Intuition : le modèle apprend que le bord gauche d'une boîte est à "environ 45 pixels", avec une distribution gaussienne centrée sur 45 — plus robuste qu'une valeur scalaire précise.

Calcul numérique

Supposons $y = 45{,}3$ (position cible d'un bord), $y_i = 45$, $y_{i+1} = 46$, et le modèle prédit $S_{45} = 0{,}72$ et $S_{46} = 0{,}28$:

$$\begin{aligned}\mathrm{DFL} &= -\big[(46 - 45{,}3)\log(0{,}72) + (45{,}3 - 45)\log(0{,}28)\big] \\ &= -\big[0{,}7 \times (-0{,}3285) + 0{,}3 \times (-1{,}2730)\big] \\ &= -\big[-0{,}2300 - 0{,}3819\big] = \mathbf{0{,}612}\end{aligned}$$

Si le modèle était parfait ($S_{45} = 0{,}7$, $S_{46} = 0{,}3$) :

$$\mathrm{DFL} = -\big[0{,}7 \times \log(0{,}7) + 0{,}3 \times \log(0{,}3)\big] \approx -\big[-0{,}2490 - 0{,}3612\big] = 0{,}610$$

Note

La DFL Loss est combinée à la CIoU Loss dans la Box Loss totale de YOLOv8 : $\mathcal{L}_{\mathrm{box}} = \lambda_{\mathrm{CIoU}} \cdot \mathcal{L}_{\mathrm{CIoU}} + \lambda_{\mathrm{DFL}} \cdot \mathcal{L}_{\mathrm{DFL}}$

Chapter 5

Métriques Opérationnelles

5.1 FPS — Frames Per Second

Définition

Les **FPS** mesurent le nombre d'images traitées par seconde. Indicateur clé pour les applications temps réel.

Formule

$$\text{FPS} = \frac{1}{\text{Latence (s)}} = \frac{\text{Nombre d'images}}{\text{Temps total (s)}}$$

Comparaison des modèles YOLOv8

Modèle	Latence (ms)	FPS	mAP50	Params
YOLOv8n	1.47	680	37.3	3.2M
YOLOv8s	2.66	376	44.9	11.2M
YOLOv8m	5.86	171	50.2	25.9M
YOLOv8l	9.06	110	52.9	43.7M
YOLOv8x	14.37	70	53.9	68.2M

Source : Ultralytics benchmark, GPU Tesla T4, image 640×640.

5.2 GFLOPs — Giga Floating Point Operations

Définition

Les **GFLOPs** mesurent la complexité computationnelle d'une passe avant (forward pass) du modèle. Indicateur de la charge de calcul requise.

Formule approximative (couche convolutive)

$$\text{FLOPs}_{\text{conv}} = 2 \times C_{\text{in}} \times C_{\text{out}} \times K_w \times K_h \times H_{\text{out}} \times W_{\text{out}}$$

Le facteur 2 représente une multiplication et une addition.

Exemple de calcul sur une couche

Couche convolutive : $C_{\text{in}} = 64$, $C_{\text{out}} = 128$, $K = 3\times3$, sortie $H_{\text{out}}\times W_{\text{out}} = 80\times80$:

$$\text{FLOPs} = 2\times64\times128\times3\times3\times80\times80 = 2\times64\times128\times9\times6400 \approx 943\,\text{M FLOPs} = 0{,}94\ \text{GFLOPs}$$

5.3 Paramètres du modèle (Params)

Définition

Le nombre de **paramètres** (poids) détermine la capacité d'apprentissage du modèle et son empreinte mémoire.

Formule (couche convolutive)

$$\text{Params}_{\text{conv}} = C_{\text{out}} \times (C_{\text{in}} \times K_w \times K_h + 1)$$

Le $+1$ représente le biais.

Calcul

Pour la couche de l'exemple précédent :

$$\text{Params} = 128 \times (64 \times 3 \times 3 + 1) = 128 \times 577 = 73\,856 \approx 74\text{k paramètres}$$

Chapter 6

Métriques de Diagnostic

6.1 Confidence Score

Définition

Le **score de confiance** est la probabilité jointe qu'une région contient un objet *et* que cet objet appartient à une classe donnée.

Formule

$$\text{Conf} = P(\text{object}) \times P(\text{class} \mid \text{object}) = P(\text{object}) \times P_c$$

Exemple

- $P(\text{object}) = 0{,}92$ — le modèle est sûr à 92% qu'il y a un objet.
- $P(\text{voiture} \mid \text{objet}) = 0{,}88$ — parmi les objets, 88% de chances que ce soit une voiture.
- $\text{Conf} = 0{,}92 \times 0{,}88 = \mathbf{0{,}81}$

Cette détection passe le seuil de confiance par défaut de 0,25.

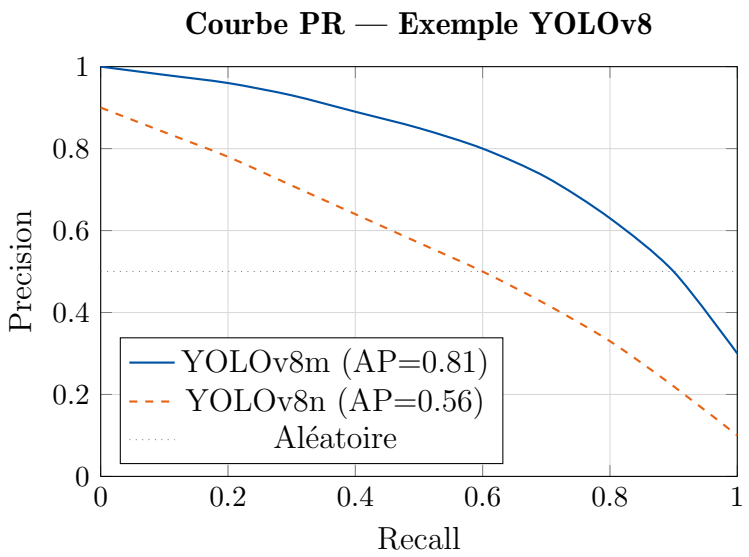
6.2 Courbe PR (Precision-Recall)

Définition

La **courbe PR** trace la Precision en fonction du Recall lorsqu'on fait varier le seuil de confiance de 1,0 (très restrictif) à 0,0 (très permissif).

Interprétation :

- Courbe proche du coin supérieur droit ($R = 1, P = 1$) : modèle excellent.
- Aire sous la courbe = **AP** (voir section précédente).
- Chute rapide de la Precision quand le Recall augmente : le modèle produit beaucoup de fausses alarmes pour "capturer" les derniers objets.



6.3 Courbe F1-Confidence

Définition

La **courbe F1-Confidence** affiche le F1-Score en fonction du seuil de confiance. Elle permet d'identifier le seuil optimal à utiliser lors de l'inférence.

Tableau d'analyse

Seuil conf.	P	R	F1	Recommandation
0.10	0.61	0.95	0.745	Trop permissif
0.25	0.72	0.88	0.792	Défaut YOLOv8
0.40	0.81	0.79	**0.800**	**Optimal**
0.55	0.89	0.68	0.771	Précis, moins de rappel
0.70	0.94	0.52	0.671	Très restrictif
0.85	0.98	0.31	0.471	Trop restrictif

Le seuil optimal est 0,40, maximisant le F1-Score à 0,800.

6.4 Matrice de Confusion

Définition

La **matrice de confusion** est un tableau $N \times N$ (où N est le nombre de classes + 1 pour "background") croisant classes prédites et classes réelles. Elle révèle les confusions inter-classes et les taux de détection manquée.

Matrice de confusion — 3 classes

Normalisée par rapport au total des GT par classe :

	Prédit			
Réel	Voiture	Piéton	Vélo	Manqué
Voiture	**0.89**	0.03	0.02	0.06
Piéton	0.05	**0.76**	0.08	0.11
Vélo	0.02	0.12	**0.71**	0.15

Lecture :

- 89% des voitures réelles sont détectées comme "voiture".
- 12% des vélos sont confondus avec des piétons.
- 15% des vélos sont manqués (FN).

Chapter 7

Tableau Récapitulatif Exhaustif

Métrique	Formule	Plage	Rôle
Precision (P)	$\frac{TP}{TP+FP}$	$[0,1]$, $\uparrow$	Taux de détections correctes
Recall (R)	$\frac{TP}{TP+FN}$	$[0,1]$, $\uparrow$	Taux d'objets réels détectés
F1-Score	$2 \cdot \frac{P \cdot R}{P+R}$	$[0,1]$, $\uparrow$	Équilibre P/R, choix du seuil
AP (par classe)	$\int_0^1 P(R)\,dR$	$[0,1]$, $\uparrow$	Précision globale d'une classe
mAP50	$\frac{1}{N}\sum AP_i^{@0.5}$	$[0,1]$, $\uparrow$	Métrique principale YOLOv8
mAP50-95	$\frac{1}{10}\sum_t \mathrm{mAP}_t$	$[0,1]$, $\uparrow$	Standard COCO, localisation fine
IoU	$\frac{\lvert B_p \cap B_g \rvert}{\lvert B_p \cup B_g \rvert}$	$[0,1]$, $\uparrow$	Qualité de localisation
GIoU	$\mathrm{IoU} - \frac{\lvert C \setminus \mathrm{union} \rvert}{\lvert C \rvert}$	$[-1,1]$, $\uparrow$	IoU + pénalité distance
DIoU	$\mathrm{IoU} - \frac{\rho^2}{c^2}$	$[-1,1]$, $\uparrow$	IoU + distance des centres
CIoU	$\mathrm{DIoU} - \alpha v$	$[-1,1]$, $\uparrow$	Loss de localisation YOLOv8
Box Loss	$1 - \mathrm{CIoU}$	$[0,+\infty)$, $\downarrow$	Erreur de régression des boîtes

Métrique	**Formule**	**Plage**	**Rôle**
Cls Loss	BCE par classe	$[0, +\infty)$, $\downarrow$	Erreur de classification
DFL Loss	$-[(\cdot)\log S_i + (\cdot)\log S_{i+1}]$	$[0, +\infty)$, $\downarrow$	Précision des bords (distribution)
FPS	1/latence	$\mathbb{R}^+$, $\uparrow$	Vitesse temps réel
GFLOPs	Opérations flottantes	$\mathbb{R}^+$, $\downarrow$	Charge computationnelle
Params (M)	Nb de poids du modèle	$\mathbb{R}^+$, contexte	Taille et capacité du modèle
Confidence	$P(\text{obj}) \times P_c$	$[0, 1]$, $\uparrow$	Certitude d'une détection
Courbe PR	$P = f(R)$	Visualisation	Trade-off P/R global
Courbe F1-Conf	$F1 = f(\text{conf})$	Visualisation	Seuil de confiance optimal
Matrice confusion	$N \times N$ classes	Visualisation	Confusions inter-classes

$\uparrow$ *: une valeur plus élevée est meilleure.* $\downarrow$ *: une valeur plus basse est meilleure.*

Chapter 8

Implémentation Python de Référence

Listing 8.1: Calcul des métriques de base avec NumPy

```
import numpy as np

#
# 1. M triques de classification
#

TP, FP, FN = 6, 3, 2

precision = TP / (TP + FP)
recall    = TP / (TP + FN)
f1        = 2 * precision * recall / (precision + recall)

print(f"Precision : {precision:.4f}")  # 0.6667
print(f"Recall    : {recall:.4f}")     # 0.7500
print(f"F1-Score  : {f1:.4f}")         # 0.7059

#
# 2. Calcul de l'IoU
#

def compute_iou(box_pred, box_gt):
```

```
    """
    box format: (x1, y1, x2, y2)
    """
    x1 = max(box_pred[0], box_gt[0])
    y1 = max(box_pred[1], box_gt[1])
    x2 = min(box_pred[2], box_gt[2])
    y2 = min(box_pred[3], box_gt[3])

    inter = max(0, x2 - x1) * max(0, y2 - y1)
    area_pred = (box_pred[2]-box_pred[0]) * (box_pred[3]-box_pred[1])
    area_gt   = (box_gt[2]-box_gt[0])   * (box_gt[3]-box_gt[1])
    union = area_pred + area_gt - inter

    return inter / union if union > 0 else 0.0

iou = compute_iou((100,150,250,350), (120,160,270,370))
print(f"IoU        : {iou:.4f}")  # 0.6712

#


# 3. Calcul de l'AP (m thode des 11 points)
#


def compute_ap_11pts(precisions, recalls):
    """
    precisions, recalls : listes tri es par confiance d croissante
    """
    ap = 0.0
    for t in np.linspace(0, 1, 11):
        # Precision max pour Recall >= t
        p_interp = max(
            [p for p, r in zip(precisions, recalls) if r >= t],
            default=0
        )
        ap += p_interp / 11
    return ap

```

```python
prec = [1.0, 1.0, 0.667, 0.75, 0.6, 0.667, 0.571, 0.625, 0.556, 0.5]
rec  = [0.2, 0.4, 0.4,   0.6,  0.6, 0.8,   0.8,   1.0,   1.0, 1.0]
ap   = compute_ap_11pts(prec, rec)
print(f"AP (11pts): {ap:.4f}")  #     0.5848

#


# 4. Entra nement YOLOv8 avec Ultralytics
#


from ultralytics import YOLO

model = YOLO("yolov8m.pt")           # Charger le mod le pr - entra n

results = model.train(
    data   = "coco128.yaml",         # Fichier de configuration du dataset
    epochs = 100,
    imgsz  = 640,
    batch  = 16,
    device = 0                       # GPU 0
)

# 5.  valuation
metrics = model.val(data="coco128.yaml", imgsz=640)
print(f"mAP50    : {metrics.box.map50:.4f}")
print(f"mAP50-95 : {metrics.box.map:.4f}")
print(f"Precision: {metrics.box.mp:.4f}")
print(f"Recall   : {metrics.box.mr:.4f}")
```

Chapter 9

Conclusion et Guide Pratique

9.1 Quelle métrique utiliser selon l'objectif ?

Objectif	Métrique prioritaire	Secondaire
Comparer des modèles	mAP50-95 (COCO)	mAP50
Déploiement rapide	FPS / Latence	GFLOPs
Minimiser fausses alarmes	Precision	F1-Score
Minimiser objets manqués	Recall	F1-Score
Équilibre général	F1-Score / mAP50	Courbe PR
Analyser les erreurs	Matrice de confusion	Courbe F1-Conf
Localisation précise	mAP50-95 / CIoU	DIoU
Embarqué / Edge AI	Params (M) / Size	GFLOPs

9.2 Seuils de référence YOLOv8 (COCO)

Modèle	mAP50	mAP50-95	Params	FPS (T4)
YOLOv8n	52.9	37.3	3.2M	680
YOLOv8s	61.8	44.9	11.2M	376
YOLOv8m	67.2	50.2	25.9M	171
YOLOv8l	69.8	52.9	43.7M	110
YOLOv8x	71.0	53.9	68.2M	70

9.3 Checklist d'évaluation YOLOv8

Checklist : évaluer un modèle YOLOv8

- ☐ Vérifier la convergence des **3 losses** (box, cls, dfl) sur train *et* val.
- ☐ Contrôler l'absence de **surapprentissage** : val_loss $\leq 1{,}5\times$ train_loss.
- ☐ Comparer **mAP50** et **mAP50-95** : écart $> 0{,}25$ = localisation imprécise.
- ☐ Analyser la **matrice de confusion** pour détecter les confusions inter-classes.
- ☐ Utiliser la **courbe F1-Confidence** pour choisir le seuil de confiance optimal.
- ☐ Vérifier **AP par classe** : classes avec AP $< 0{,}5$ nécessitent plus de données.
- ☐ Mesurer les **FPS** sur le matériel cible.

Note

Ce document couvre l'ensemble des métriques utilisées par YOLOv8 dans le contexte de la détection et de la reconnaissance d'objets. Pour des cas d'usage spécifiques (segmentation, pose estimation), des métriques supplémentaires comme le **Mask AP** ou l'**OKS (Object Keypoint Similarity)** entrent en jeu, mais restent des extensions des principes présentés ici.